Swing GUI Programming and JDBC Connectivity

Object-Oriented Programming with Java — Week 9

Index

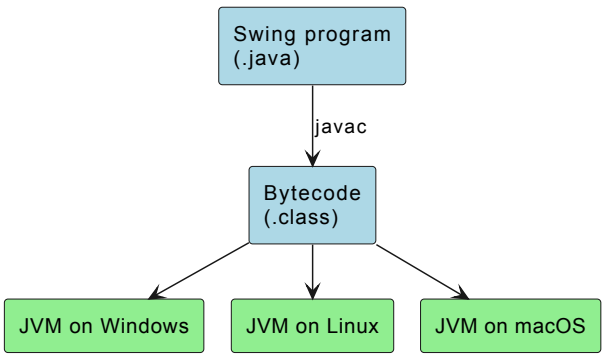
- Swing Fundamentals — escaping the black-and-white console
- Swing Components — the parts catalogue: windows, labels, buttons and friends
- Creating a Window with JFrame — "Hello, window!"
- Adding Components to a Frame — furnishing the empty room
- Event Handling — teaching buttons to do something besides look pretty
- Event Delegation Model — source, event, listener: the bell-and-waiter system
- ActionListener Example — your first working click
- Common Event Listeners — who listens to the mouse, keyboard and window
- Layout Managers — letting Java arrange the furniture
- Building a Simple GUI Application — a calculator that can (so far) only add
- Swing Summary and Exercise Themes — what to take away and what to build
- JDBC Connectivity — making Java talk to a database
- JDBC Architecture and Steps — the chain of command and the seven-step recipe
- JDBC Connection Example — knocking on the database door
- CRUD Operations with JDBC — create, read, update, delete
- PreparedStatement — the safer, tidier way to send queries
- JDBC Summary and Exercise Themes — recap and practice ideas
- Practice Questions — five problems to test yourself

Swing Fundamentals

The Problem

Every program so far has talked to the user through a console: the user types text, the program prints text. Real desktop software (a calculator, a login screen, a billing window) has windows, buttons, text boxes and menus. We need a way to build these in Java.

Swing is a Java **GUI (Graphical User Interface) toolkit** used for developing desktop applications. Think of the screen of a bank ATM: a display window, labels telling you what to do, a keypad for input and buttons to confirm. Swing is the box of ready-made parts for building such a screen. Because Swing programs run on the JVM, the same compiled bytecode runs on Windows, Linux or macOS, which is what "platform independent" means.



Learning Objectives

After this module you should be able to:

- Develop a graphical user interface application using Java Swing (text fields, buttons, check boxes and so on).
- Handle user events, such as a button click, using event listeners.
- Use different layout managers to arrange GUI components.
- Build a simple interactive desktop application, such as a calculator with buttons and a text field for input and output.

Swing can be used for any kind of desktop application: a calculator, a game, a reservation system and more.

The Package

All Swing classes live in the package `javax.swing`. To use them we must import it.

```
import javax.swing.*;
```

Part	Meaning
`import`	Tells the compiler to make classes from another package usable by their short names
`javax.swing`	The package ("x" stands for *extension*) that holds Swing classes such as `JFrame` and `JButton`
`.*`	Import every class directly inside this package
`;`	Ends the statement

> **Note for Review:** `import javax.swing.*;` imports only the classes directly inside `javax.swing`. It does **not** import sub-packages. That is why event handling needs a separate `import java.awt.event.*;`. If you forget an import, the compiler reports that it cannot find the symbol (for example, `JFrame`).

Features of Swing

Feature	What it means	Why it helps
Platform independent	The same program runs on any operating system with a JVM	Write once, run on Windows or Linux alike
Rich GUI components	A large ready-made set: buttons, text boxes, radio buttons, combo boxes, etc.	You do not draw everything yourself
Event-driven programming	The program reacts to actions such as single click, double click, right or left click	Clicking can send, store or verify information
Lightweight components	Components are drawn by Java itself instead of depending on the operating system's native widgets (the older AWT widgets are "heavyweight")	Less overhead and a consistent look

---

Swing Components

Every GUI element is an **object of a Swing class**. The class names start with `J` (JFrame, JLabel, ...), which distinguishes them from the older AWT classes (`Frame`, `Label`, ...).

Component	Purpose	Everyday example
`JFrame`	Main window	The calculator window itself
`JLabel`	Display text	The word "Username" next to a box
`JButton`	Button	"Login", "Add", "Click"
`JTextField`	Single-line input field	Entering your name or a number
`JTextArea`	Multiple lines of text	The writing area of a notepad
`JCheckBox`	Checkbox	"Remember me"
`JRadioButton`	Radio option (pick one of a group)	Gender or payment method
`JComboBox`	Dropdown	Choosing a branch from a list

There are more components than these; the table lists the common ones used in this lecture.

Supplementary: how each one is created (all are ordinary constructor calls).

```
JLabel userLabel = new JLabel("Username");
JTextField nameField = new JTextField();
JTextArea notesArea = new JTextArea();
JCheckBox rememberBox = new JCheckBox("Remember me");
JRadioButton maleOption = new JRadioButton("Male");

String[] branches = {"CSE", "ECE", "ME"};
JComboBox<String> branchBox = new JComboBox<>(branches);
```

Creating a Window with JFrame

The Problem

Before anything else, we need a main window to hold everything. `JFrame` is a **predefined class** in the Swing package. Creating a `JFrame` object gives us a window.

The smallest Swing program imports the package, creates one `JFrame` object, sets its size and makes it visible. When it runs, **a window titled "My Application" appears**. The size is in **pixels**: 400 wide and 300 high. You can change it to 200 by 300 or 500 by 600 as you like.

```
import javax.swing.*;

class MyFrame {
    public static void main(String args[]) {
        JFrame frame = new JFrame("My Application");
        frame.setSize(400, 300);
        frame.setVisible(true);
    }
}
```

General Pattern

```
JFrame variableName = new JFrame("Window title");
variableName.setSize(width, height);
variableName.setVisible(true);
```

Syntax Breakdown

Code	Meaning
`JFrame frame`	**Declares a reference variable** named `frame` that can refer to a `JFrame` object. No window exists yet.
`new JFrame("My Application")`	`new` allocates a new object; the **constructor** `JFrame(String title)` runs and sets the title. `new` returns a **reference** to the object.
`JFrame frame = new JFrame(...)`	The returned reference is stored in `frame`. Declaration and creation are combined.
`frame.setSize(400, 300)`	Method call: width 400, height 300 pixels
`frame.setVisible(true)`	Shows the window. With `false` it stays hidden.

The same thing in two steps shows the difference between declaration and creation:

```
JFrame frame;                          // declaration: only a reference variable
frame = new JFrame("My Application");  // creation and assignment
```

Execution Trace

Step	Operation	Result
1	`new JFrame("My Application")`	A frame object is created with the title set
2	Reference stored in `frame`	`frame` now refers to that object
3	`setSize(400, 300)`	The object's size is changed (still not on screen)
4	`setVisible(true)`	The window appears on the screen

> **Note for Review:** Common mistakes: (1) forgetting `setVisible(true)`, so nothing appears; (2) forgetting `setSize(...)`, so the window is tiny (default size is 0 by 0). **Supplementary:** by default, closing the window only hides it and the program may keep running; adding `frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);` makes the program end when the window is closed.

Adding Components to a Frame

The Problem

An empty window is useless. A real application, such as a login screen, needs a label, a text box and a button inside the window.

The Program: Login Screen

This program creates a login window. (The lecture uses the short names `f`, `l`, `t`, `b`; here they are renamed so the code explains itself: `frame`, `userLabel`, `userField`, `loginButton`.)

```
import javax.swing.*;

class Demo {
    public static void main(String args[]) {
        JFrame frame = new JFrame("Login");
        JLabel userLabel = new JLabel("Username");
        JTextField userField = new JTextField();
        JButton loginButton = new JButton("Login");

        userLabel.setBounds(50, 50, 100, 30);
        userField.setBounds(150, 50, 150, 30);
        loginButton.setBounds(100, 100, 100, 30);

        frame.add(userLabel);
        frame.add(userField);
        frame.add(loginButton);

        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

Output: a window titled "Login" with the text "Username", an empty text box beside it, and a "Login" button below them.

Positioning with setBounds

`setBounds(x, y, width, height)` decides **where a component goes and how big it is**. The coordinates are measured from the **top-left corner** of the frame: `x` grows to the right and `y` grows **downward**. Choosing the bounds lets you place a component on the left, in the middle, or on the right. Notice that the label ends at x = 150 exactly where the text field begins, so they sit side by side. (The title bar and borders use some of the 400 by 300 frame, so the usable area is slightly smaller.)

Component	x	y	Width	Height	Horizontal span
`userLabel`	50	50	100	30	50 to 150
`userField`	150	50	150	30	150 to 300
`loginButton`	100	100	100	30	100 to 200

Syntax Breakdown

Code	Meaning
`frame.add(userLabel)`	Adds the component to the frame. `frame` is the container; `userLabel` is the thing placed inside.
`frame.setLayout(null)`	Turns off automatic arrangement, so our `setBounds` values are obeyed (**absolute positioning**)
`new JTextField()`	Creates an empty input box
`new JLabel("Username")`	Creates a label with the given text

Execution Trace

Step	Operation	Result
1	Create frame, label, text field, button	Four objects exist, none shown yet
2	`setBounds(...)` on three components	Each knows its position and size
3	`frame.add(...)` three times	The components belong to the frame
4	`setSize`, `setLayout(null)`, `setVisible(true)`	The window appears with all three components at their exact positions

> **Note for Review:** A very common beginner mistake is calling `setBounds` but forgetting `setLayout(null)`. The frame's default layout manager then decides the positions and ignores your numbers, so the form will not appear as you positioned it. You may add as many components as the application needs; the three here are just an example.

Event Handling

The Problem

Our login window appears, but clicking "Login" does nothing. A real program must react: send data, open a new page, store a record, or verify credentials.

Event handling defines how a program responds to **user actions**. Examples of events:

- **Button click**: click the button to perform something.
- **Mouse movement**: moving the mouse over a component for some purpose.
- **Keyboard input**: typing values, for example two numbers to add.

Without event handling a GUI is only a picture. With it, the application becomes **event-driven**: it waits for the user to do something and then responds.

Event Delegation Model

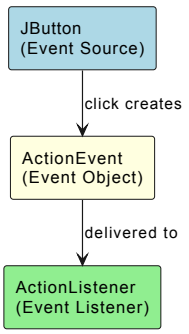
The Intuition and the Analogy

Imagine a restaurant. A customer presses the call bell on the table (**the source of the signal**). The bell sends a small signal that says which table pressed it (**the event information**). A waiter who has **registered** for that table hears it and responds (**the listener**).

The button does not handle the click itself. It **delegates** the job to a separate listener object. This is why the approach is called the **event delegation model**. (Where the analogy stops: in Java, the listener must be registered with the source before the event happens, and one source may have several listeners.)

The Three Components

Component	Meaning	In the lecture's example
Event source	The component where the event happens	`JButton`
Event object	An object holding information about the event	`ActionEvent`
Event listener	The object that is notified and contains the handling code	`ActionListener`



ActionListener Example

The Goal

Show a message box saying "Button Clicked" whenever the user clicks a button. Two packages are needed: `javax.swing.*` for the components, and `java.awt.event.*` for event handling (`ActionListener` and `ActionEvent` live there).

```
import javax.swing.*;
import java.awt.event.*;

class ButtonDemo {
    public static void main(String args[]) {
        JFrame frame = new JFrame();
        JButton clickButton = new JButton("Click");

        clickButton.setBounds(100, 100, 100, 40);

        clickButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button Clicked");
            }
        });

        frame.add(clickButton);
        frame.setSize(300, 300);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

Output: a 300 by 300 window with a "Click" button. Clicking it opens a message dialog containing "Button Clicked".

The class is saved as `ButtonDemo.java` by convention (the file name must match the class name only for a `public` class).

Syntax Breakdown

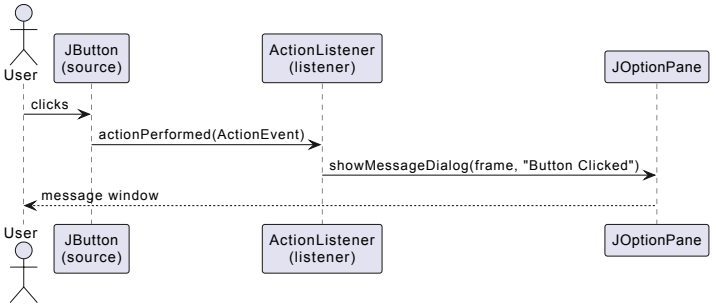
General pattern for registering a listener:

```
sourceComponent.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        // code to run when the action happens
    }
});
```

Piece	Meaning
`addActionListener(...)`	Method of the button that **registers** a listener with it. Until a listener is registered, the click is ignored.
`ActionListener`	A predefined **interface** with one method, `actionPerformed`.
`new ActionListener() { ... }`	An **anonymous inner class**: it creates a nameless class that implements the interface and immediately creates one object of it. The listener body is written right where it is needed.
`public void actionPerformed(ActionEvent e)`	The method that Java **calls automatically** when the button is clicked. The signature must match exactly. `e` carries the event information.
`JOptionPane.showMessageDialog(frame, "Button Clicked")`	Static method that shows a pop-up message. The first argument is the parent component (the dialog appears over the frame); the second is the text.
`});`	Closes, in order: the anonymous class body `}`, the argument list `)`, and the statement `;`

Execution Trace

When the program runs, `main` finishes setting up and the window stays on screen, waiting. Nothing inside `actionPerformed` runs until the user clicks. Each click creates a new `ActionEvent` and Java calls `actionPerformed` once. This is the essence of event-driven programming: **you write the response; Java decides when to call it**.



Step	Operation	Result
1	Create `JFrame` and `JButton`; set bounds	Objects ready
2	`addActionListener(...)`	Listener object registered with the button
3	`add`, `setSize`, `setLayout(null)`, `setVisible(true)`	Window appears; program waits
4	User clicks "Click"	`ActionEvent` created and passed to `actionPerformed`
5	`showMessageDialog(...)` runs	"Button Clicked" dialog appears

> **Note for Review:** Frequent mistakes: forgetting `import java.awt.event.*;`; writing the method as `void actionPerformed(...)` without `public` (an interface method must be implemented as `public`); using a different parameter type. Also, a local variable such as `frame` used inside the anonymous class must be **effectively final**, meaning it is never reassigned after initialization. **Supplementary:** since `ActionListener` has a single method, Java 8 and later also allow a shorter lambda form: `clickButton.addActionListener(e -> JOptionPane.showMessageDialog(frame, "Button Clicked"));`.

Common Event Listeners

Different kinds of events have different listener interfaces.

Listener	Event type	Registered with
`ActionListener`	Button actions	`addActionListener(...)`
`MouseListener`	Mouse events	`addMouseListener(...)`
`KeyListener`	Keyboard events	`addKeyListener(...)`
`WindowListener`	Window events	`addWindowListener(...)`

> **Note for Review (supplementary):** `ActionListener` has only one method, but `MouseListener`, `KeyListener` and `WindowListener` each declare several, and an implementing class must define all of them (or extend the matching *adapter* class). Also, the lecture lists "mouse movement" as an example event: strictly, pressing, releasing and clicking are handled by `MouseListener`, while moving or dragging is handled by `MouseMotionListener`.

Layout Managers

The Problem

With `setLayout(null)` every component has a fixed position. If the window is resized, or the program runs on a screen with different settings, the design can look broken. We want the arrangement to adapt automatically.

Analogy: instead of telling each guest at an event the exact coordinates of their chair, you tell a manager "arrange the guests in rows" and the manager seats everyone, however many arrive. A **layout manager** does this for components.

Definition: layout managers **automatically arrange components inside containers**.

Advantages:

- Automatic resizing
- Platform independence (no pixel-perfect assumptions)
- Better GUI design

General Syntax

```
container.setLayout(new LayoutManagerName());
```

`setLayout` is called on the container (for example, the frame) and receives a **layout manager object** created with `new`. The lecture writes `setLayout(new FlowLayout());`, meaning "call this on the current container". `FlowLayout`, `BorderLayout`, `GridLayout` and `CardLayout` belong to `java.awt`, so add `import java.awt.*;`. (`BoxLayout` belongs to `javax.swing`.)

Layout	Arrangement	Typical use
`FlowLayout`	Left to right	Rows of buttons
`BorderLayout`	Five regions: North, South, East, West, Center	Overall window structure
`GridLayout`	Rows and columns	Calculator keypad
`CardLayout`	One component at a time	Multi-page forms
`BoxLayout`	Vertical or horizontal stack	Vertical menus

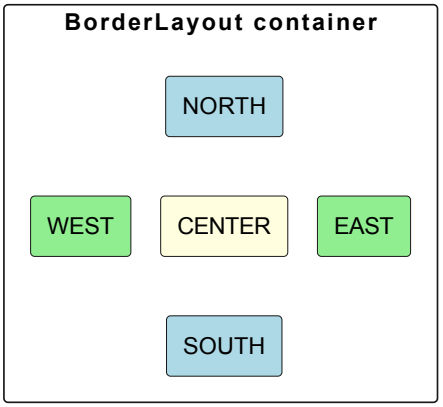
FlowLayout

Arranges components **from left to right** in the order they are added, moving to the next row when a row is full.

```
setLayout(new FlowLayout());
```

BorderLayout

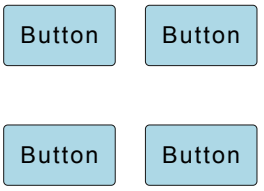
`BorderLayout` divides the container into **five regions**: **North** at the top, **South** at the bottom, and **West**, **Center** and **East** across the middle. Create it with `setLayout(new BorderLayout());` and add a component to a region by name, for example `frame.add(button, BorderLayout.NORTH);`. Components added without a region go to the Center.



> **Note for Review (supplementary):** `JFrame` uses `BorderLayout` by default, and `JPanel` uses `FlowLayout` by default. That is why components pile into the Center unless you change the layout.

GridLayout

`GridLayout` arranges components **in rows and columns** of equal-sized cells. `new GridLayout(2, 2)` means 2 rows and 2 columns, so four buttons appear as two rows of two buttons. Components fill the grid row by row, left to right.



Supplementary: a complete runnable example of the lecture's `GridLayout(2, 2)` output.

```
import javax.swing.*;
import java.awt.*;

class GridDemo {
    public static void main(String args[]) {
        JFrame frame = new JFrame("Grid Demo");
        frame.setLayout(new GridLayout(2, 2));
        frame.add(new JButton("Button"));
        frame.add(new JButton("Button"));
        frame.add(new JButton("Button"));
        frame.add(new JButton("Button"));
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

CardLayout and BoxLayout

- **CardLayout:** displays **one component at a time**, like flipping through cards one by one. Created with `setLayout(new CardLayout());` and used for multi-page forms and wizard-style applications.
- **BoxLayout:** arranges components **vertically or horizontally**. Unlike the others, its constructor needs the container: `new BoxLayout(container, BoxLayout.Y_AXIS)` stacks vertically; `BoxLayout.X_AXIS` stacks horizontally.

> **Note for Review:** `setLayout(null)` (used in the lecture's examples) is *not* a layout manager. It means "no manager; I place everything myself with `setBounds`". It is simple for learning, but it loses the automatic resizing advantage described above.

Building a Simple GUI Application

The Goal

Build a **calculator** that performs addition. Functionality:

1. Take two numbers as input in two text fields.
2. When the button is clicked, add them (this is why an action listener is needed).
3. Display the result in a label.

For example, with 10 and 20 entered:

$$\text{Sum} = a + b = 10 + 20 = 30$$

Part 1: Components and Positions

The program uses `javax.swing.*` and `java.awt.event.*`. These statements are consecutive lines inside `main`.

```
import javax.swing.*;
import java.awt.event.*;

class Calculator {
    public static void main(String args[]) {
        JFrame frame = new JFrame("Calculator");

        JTextField firstField = new JTextField();
        JTextField secondField = new JTextField();
        JButton addButton = new JButton("Add");
        JLabel resultLabel = new JLabel();

        firstField.setBounds(50, 50, 100, 30);
        secondField.setBounds(50, 100, 100, 30);
        addButton.setBounds(50, 150, 100, 30);
        resultLabel.setBounds(50, 200, 200, 30);
```

Part 2: The Click Handler

```
        addButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                int first = Integer.parseInt(firstField.getText()
                int second = Integer.parseInt(secondField.getText
                resultLabel.setText("Sum = " + (first + second));
            }
        });
```

Part 3: Assemble and Show

```
        frame.add(firstField);
        frame.add(secondField);
        frame.add(addButton);
        frame.add(resultLabel);

        frame.setSize(300, 300);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

(The lecture uses `t1`, `t2`, `add`, `result`, `a` and `b`; the names are expanded here for readability.)

Syntax Breakdown

Code	Meaning
`firstField.getText()`	Returns the text typed in the box. **It is always a `String`**, even if the user types digits.
`Integer.parseInt(String)`	Converts a text such as `"10"` into the `int` value 10
`resultLabel.setText(...)`	Changes the text displayed by the label
`"Sum = " + (first + second)`	The parentheses make Java add the numbers first, then join the result to the text

Why the parentheses matter: without them, `"Sum = " + first + second` is evaluated left to right as text joining, producing `Sum = 1020` for inputs 10 and 20 (a logically incorrect output, not a compile error).

The variables `firstField`, `secondField` and `resultLabel` are used inside the anonymous class, so they must be effectively final (they never change after creation).

Execution Trace (inputs 10 and 20)

Step	Operation	Result
1	Window shown	Two empty boxes, "Add" button, empty label
2	User types 10 and 20	Text boxes hold `"10"` and `"20"`
3	User clicks "Add"	`actionPerformed` is called
4	`parseInt` on both texts	`first = 10`, `second = 20`
5	`setText("Sum = " + 30)`	Label shows `Sum = 30`

> **Note for Review:** If a box is empty or contains non-numeric text, `Integer.parseInt` throws a `NumberFormatException` at **runtime** (the code compiled fine), and the label is not updated. Real applications validate the input first.

Extending the Calculator

This calculator only adds. To support other operations, use one pair of input boxes and one result label, but give each operation its own button with its own listener: Add performs `first + second`, Subtract performs `first - second`, Multiply `first * second`, Divide `first / second`. The logic of each operation goes inside that button's `actionPerformed`. For example:

```
subtractButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        int first = Integer.parseInt(firstField.getText());
        int second = Integer.parseInt(secondField.getText());
        resultLabel.setText("Difference = " + (first - second));
    }
});
```

> **Note for Review (supplementary):** with `int` values, division truncates (7 / 2 gives 3) and dividing by 0 throws an `ArithmeticException`. Use `double` and check for zero when building a real division button.

Swing Summary and Exercise Themes

- Swing components (`JFrame`, `JButton`, `JLabel`, `JTextField`, `JRadioButton`, ...) are used to build GUI applications.
- User events (clicks, mouse movement, keyboard input) are handled through the event delegation model with listeners.
- Layout managers give better design, automatic resizing and platform independence.
- These tools let us build interactive Java applications for calculation, reservation, gaming, and more.

Exercise themes from the lecture:

- A Swing login form using `JTextField` and `JButton`.
- A GUI calculator with addition, subtraction, multiplication, division and percentage.
- A student registration form (student name, roll number, branch) using event handling.
- A demonstration of different layout managers using multiple buttons.
- A simple notepad application using `JTextArea` and file handling.

JDBC Connectivity

The Problem

A student-record program must keep data (IDs, names, marks) somewhere permanent. That "somewhere" is usually a **database**, which lives outside the Java program. We need a standard way for Java to connect to it, retrieve information, update it and delete it.

Analogy: you speak English, the database speaks its own language. You need an interpreter. Java calls a standard set of methods, and a **driver** translates them for the specific database. (Where the analogy stops: the driver also handles the network communication with the database.)

Definition

JDBC (Java Database Connectivity) is an **API** (Application Programming Interface) that allows Java applications to **connect and interact with databases**. Its classes are in the package `java.sql`.

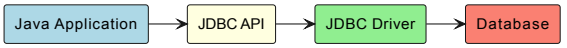
Learning objectives: connect a Java application to a database using JDBC, and perform the basic **CRUD** operations (Create, Read, Update, Delete).

Common databases: MySQL, PostgreSQL, Oracle, SQLite.

JDBC Architecture and Steps

Architecture

A Java application never talks to the database directly. It calls the **JDBC API**; the API uses a **JDBC driver** that knows how to talk to one particular database; the driver stores information in or retrieves information from the **database**. The same chain is used for storing and retrieving.



The Seven Steps

Every JDBC program follows the same sequence, and the order matters: the connection must exist before a statement can be created, and the statement must exist before a query can run. Just as earlier topics needed their own packages (for example the Swing packages), JDBC needs the special package `java.sql`.



Step	Typical code
1. Import JDBC package	`import java.sql.*;`
2. Load JDBC driver	`Class.forName("com.mysql.cj.jdbc.Driver");`
3. Establish connection	`Connection con = DriverManager.getConnection(url, user, password);`
4. Create statement	`Statement stmt = con.createStatement();`
5. Execute query	`stmt.executeQuery(sql)` or `stmt.executeUpdate(sql)`
6. Process results	`while (rs.next()) { ... }`
7. Close connection	`con.close();`

A query means a request to the database, such as "find students with the highest marks" or "check the CGPA".

JDBC Connection Example

The Goal

Connect to a MySQL database called `testdb` and print "Connected Successfully".

```
import java.sql.*;

public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "password"
            );

            System.out.println("Connected Successfully");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

> **Note:** the MySQL JDBC driver (Connector/J) must be added to the project **classpath**, or the driver class cannot be found.

Syntax Breakdown

Code	Meaning
`import java.sql.*;`	Makes `Connection`, `DriverManager`, `Statement`, `ResultSet` and others available
`Class.forName("com.mysql.cj.jdbc.Driver")`	Loads the MySQL driver class by its full name. The name must be exact. (Supplementary: in modern JDBC versions the driver registers itself, so this line is often optional, but the lecture includes it.)
`Connection con`	A reference variable for the open connection to the database
`DriverManager.getConnection(url, user, password)`	Static method that opens the connection and returns a `Connection`
`"jdbc:mysql://localhost:3306/testdb"`	URL: `jdbc:mysql` = protocol, `localhost` = this computer, `3306` = default MySQL port, `testdb` = database name
`"root"`, `"password"`	Database user name and password (they depend on your own computer's setup)
`con.close()`	Releases the connection
`try { } catch (Exception e)`	JDBC methods can fail (wrong password, driver not found). `catch` handles the error; `System.out.println(e)` prints it.

Execution

Case	What happens
Driver, URL, user and password are correct	`getConnection` succeeds; **Connected Successfully** is printed
Anything is wrong	An exception is thrown, jumps to `catch`, and the error is printed instead

> **Note for Review:** if an exception occurs before `con.close()`, the close line is skipped and the connection stays open. **Supplementary:** `try-with-resources` (`try (Connection con = ...) { ... }`) closes it automatically. Also, never hard-code real passwords in programs; `"root"` and `"password"` are placeholders.

CRUD Operations with JDBC

CRUD stands for the four basic database operations. The examples use a table `student` with two columns, `id` and `name`.

Operation	SQL command	Statement method used
Create	`INSERT`	`executeUpdate()`
Read	`SELECT`	`executeQuery()`
Update	`UPDATE`	`executeUpdate()`
Delete	`DELETE`	`executeUpdate()`

Create Operation (INSERT)

Goal: insert the record (101, Rahul) into `student`. The structure is the same as the connection example; only the query and the execute call are new.

Output: `Record Inserted` (and the table now contains one more row, with `id` 101 and `name` Rahul).

Line	Purpose
`String query = "INSERT ..."`	The SQL command stored as text. Text values inside SQL use single quotes.
`con.createStatement()`	Creates a `Statement` object that carries SQL to the database
`stmt.executeUpdate(query)`	Sends the INSERT and changes the table
`catch (Exception e)`	Handles any failure, such as a duplicate id or an unavailable database

> **Note for Review (supplementary):** `INSERT INTO student VALUES (...)` relies on the column order. Naming the columns, `INSERT INTO student (id, name) VALUES (101, 'Rahul')`, is safer.

Read Operation (SELECT)

Goal: display every row of the `student` table.

```
import java.sql.*;

public class SelectDemo {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "password"
            );

            String query = "SELECT * FROM student";

            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(query);

            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " " +
                    rs.getString("name")
                );
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output (if the table holds the Rahul record):

```
101 Rahul
```

Supplementary: the lecture assumes this table already exists, for example:

```
CREATE TABLE student (id INT PRIMARY KEY, name VARCHAR(50));
```

> **Note for Review:** `executeUpdate` is for statements that change data (INSERT, UPDATE, DELETE) and returns the number of rows affected. `executeQuery` is for SELECT and returns a `ResultSet`. Using the wrong one throws an `SQLException`.

```
import java.sql.*;

public class InsertDemo {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "password"
            );

            String query = "INSERT INTO student VALUES (101, 'Rahul')";

            Statement stmt = con.createStatement();
            stmt.executeUpdate(query);

            System.out.println("Record Inserted");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Piece	Meaning
`SELECT * FROM student`	`*` means all columns of every row
`ResultSet rs`	Holds the rows returned by the query
`rs.next()`	Moves to the next row; returns `true` if one exists, `false` when rows run out. The cursor starts *before* the first row, so `next()` must be called before reading.
`rs.getInt("id")`	Reads the `id` column of the current row as an `int`
`rs.getString("name")`	Reads the `name` column as a `String`
`... + " " + ...`	Joins id and name with a space so both print on one line

Flow of the program (the main points the lecturer highlighted, in order):

Step	Part of the program	Role
1	`import java.sql.*;`	Get the JDBC classes
2	`public class SelectDemo`, `main`	Program entry
3	`DriverManager.getConnection(...)`	Open the connection with URL, user, password
4	`String query = "SELECT * FROM student";`	Write the query
5	`createStatement()`, `executeQuery(query)`	Create the statement and run the query
6	`while (rs.next()) { ... }`	Process the results row by row
7	`con.close()`	Close the connection
8	`catch (Exception e)`	Handle errors

Update Operation (UPDATE)

To change data, write an `UPDATE` query. This one sets the name to Amit for the student with `id` 101. It runs through the same `Statement` and `executeUpdate` calls as INSERT, inside the same connection code. The `WHERE` clause chooses which row changes; without it, **every row** would be updated.

```
String query =
    "UPDATE student SET name='Amit' WHERE id=101";

Statement stmt = con.createStatement();
stmt.executeUpdate(query);
```

Delete Operation (DELETE)

`DELETE` removes rows based on a condition; it does not drop the whole table. This one deletes the row with `id` 101. As with UPDATE, a missing `WHERE` clause would delete **all rows** (the table itself would still exist).

```
String query =
    "DELETE FROM student WHERE id=101";

Statement stmt = con.createStatement();
stmt.executeUpdate(query);
```

> **Note for Review:** the UPDATE and DELETE snippets are not complete programs. They replace the query-and-execute lines of the INSERT program, inside the same `try` block, with the same connection code and `con.close()`.

PreparedStatement

The Problem

So far the SQL text was fixed. Real programs build queries from user input. If we join the input into the SQL string, a malicious user can change the meaning of the query.

Invalid / unsafe example (do not write code like this):

```
// UNSAFE: user input is pasted directly into the SQL text
String query = "SELECT * FROM student WHERE name='" + userInput + "'";
```

If the user types `' OR '1'='1`, the query becomes `... WHERE name='' OR '1'='1'`, which is true for every row. This attack is called **SQL injection**.

The Solution

A **PreparedStatement** keeps the SQL structure and the values separate. The query contains `?` **placeholders**, and the values are supplied afterwards with `setInt` and `setString`, so a value can never be interpreted as SQL. It prevents SQL injection and can also improve performance, since the same prepared statement can be executed repeatedly with different values.

```
String query =
    "INSERT INTO student VALUES (?, ?)";

PreparedStatement ps =
    con.prepareStatement(query);
ps.setInt(1, 102);
ps.setString(2, "Neha");

ps.executeUpdate();
```

Syntax Breakdown

Code	Meaning
`?`	Placeholder for a value that will be supplied later
`con.prepareStatement(query)`	Sends the SQL with placeholders to the database and returns a `PreparedStatement` (compare with `createStatement()`, which takes no SQL)
`ps.setInt(1, 102)`	Sets the **first** `?` to the integer 102
`ps.setString(2, "Neha")`	Sets the **second** `?` to the text "Neha"
`ps.executeUpdate()`	Runs the statement. **No query argument**, because the SQL was already given to `prepareStatement`.

> **Note for Review:** placeholder positions start at **1**, not 0. The setter must match the column type (`setInt` for numbers, `setString` for text). The result of the program is that a new row (102, Neha) is inserted.

JDBC Summary and Exercise Themes

JDBC is the API that lets a Java application connect and interact with a database. The workflow is: import the package, load the driver, establish the connection, create a statement, execute the query, process the results and close the connection.

Operation	SQL command
Create	`INSERT`
Read	`SELECT`
Update	`UPDATE`
Delete	`DELETE`

Exercise themes from the lecture:

- Create a MySQL database and connect to it using JDBC.
- Implement CRUD operations for the `student` table using JDBC.
- Read through the code, understand it, and analyse the results.

Practice Questions

1. **Designing a form.** Design a student registration window with a name field, a roll-number field, a branch dropdown (CSE, ECE, ME) and a "Register" button. Clicking "Register" must show a message dialog containing the name, roll number and selected branch. Write the Java code, and explain whether you would use `setLayout(null)` or a layout manager such as `GridLayout`, justifying your choice with at least two advantages or drawbacks.

2. **Predicting and fixing.** A programmer wants a button that counts its own clicks and shows the count in a label, and writes this inside `main`:

```
int clickCount = 0;
countButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        clickCount++;
        countLabel.setText("Clicks: " + clickCount);
    }
});
```

   (a) Explain why this code does not compile. (b) Fix it so the label shows `Clicks: 1`, `Clicks: 2`, ... on successive clicks, and trace the value after three clicks. (c) Describe in terms of the event delegation model what the source, event object and listener are here.

3. **Debugging JDBC.** A program is supposed to change the name of student 101 to "Amit". Identify every defect (including any that would cause unintended data changes or resource problems) and rewrite the relevant part correctly.

```
String query = "UPDATE student SET name='Amit'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(query);
System.out.println("Updated");
```

   Also explain what would happen to the table if the faulty version were somehow executed successfully.

4. **Comparing implementations.** A method must check whether a student with a given `int id` exists in the `student` table. Implementation A builds the SQL with `"SELECT * FROM student WHERE id=" + id`; implementation B uses a `PreparedStatement`. Write both versions of the query part, state which you would use for an application that accepts the student's roll number from a text field, and justify your choice by explaining what could go wrong with the other, using at least one concrete example input.